하나금융TI CoP · 프로젝트형 · AIOps

Hanati Jarvis

로그·자원을 스스로 읽고, 원인을 찾고,
운영자 승인 아래 안전하게 복구까지 끝내는 금융 IT 장애 대응 에이전트

배경 → 문제 → 해결 → 스케일업

2026.07 – 2026.10 · github.com/choimingil/hanati_jarvis

AGENDA 발표 순서와 심사 항목

“장애는 막을 수 없다. 하지만 대응 시간은 줄일 수 있다.”

01 · 배경

Why & Goal

무중단이 기본인 금융 IT,
그대로인 대응 방식,
그리고 CoP 활동 목표

약 2분 30초

① 목표달성도 ② 업무 문제

02 · 문제

Pain

탐지·분석·조치·확인
네 단계마다 생기는
지연과 위험

약 2분

② 회사성장기여도

03 · 해결

Jarvis

5가지 핵심 기능 + 시연,
기존 업무 연계,
목표 달성 결과

약 8분 · 시연 포함

④ 산출물완성도 ② 업무 연계
① 달성 결과 ⑤ 난이도·실행가능성

04 · 스케일업

Spread

팀 → 조직 → 그룹으로
확산 경로와
성과 측정 방법

약 2분 30초

③ 임팩트/확산성 ② 성장 기여

01

배경

금융 서비스는 멈추면 안 되는데,
시스템은 계속 복잡해지고 있습니다.

01 배경 금융 IT 운영 환경의 변화

서비스는 잘게 쪼개지고, 장애 신호는 여러 곳에 흩어졌습니다

24 / 7

무중단이 기본값

결제·이체·인증은 몇 분만 멈춰도 고객 거래 실패와 민원, 대외 신뢰 하락으로 바로 이어집니다. 장애 대응 시간이 곧 서비스 품질입니다.

N × M

컴포넌트 폭증

API·DB·Redis·Kafka·외부 PG·인증 서버·컨테이너가 서로 얽혀, 하나의 장애가 여러 서비스의 오류 로그로 동시에 퍼집니다.

Log · Metric

신호의 분산

애플리케이션 로그, 호스트 자원(CPU·메모리·디스크·소켓), 과거 장애 기록이 각각 다른 도구에 있어 운영자가 머릿속으로 이어 붙여야 합니다.

관측 도구는 이미 충분합니다. 부족한 것은 **“그래서 지금 무엇을 해야 하는가”**를 빠르고 안전하게 알려주는 연결 고리입니다.

장애 대응의 4단계는 여전히 사람의 손에 달려 있습니다

① 탐지
알림 수신
로그 검색 도구 접속

›

② 원인 분석
로그·지표 수동 대조
과거 사례는 담당자 기억

›

③ 조치
서버 접속 후 수동 명령
스크립트 위치 찾기

›

④ 복구 확인
“재시작했으니 됐겠지”
업무 지표 확인 누락

MTTR = 탐지 + 분석 + 조치 + 확인

대부분의 시간은 실제 조치보다 **“무엇이 원인인지, 무엇을 실행해야 하는지”**를 찾는 ②번 단계에서 소모됩니다. 숙련자가 자리에 없는 야간·주말일수록 더 길어집니다.

자동화의 딜레마

그렇다고 AI가 운영 서버를 마음대로 재시작하게 둘 수는 없습니다. 금융권에서 자동화는 **통제·감사·책임 소재**가 함께 설계되어야만 도입할 수 있습니다.

01 배경 CoP 활동 목표

그래서 우리 CoP는 네 가지 목표를 세웠습니다

GOAL 1

장애 대응 전 과정을 하나의 서비스로

수집 → 분석 → 조치 → 복구 확인까지 끊김 없는 End-to-End 파이프라인 구현

GOAL 2

금융 빈발 장애의 자동 분석·추천

DB·캐시·PG·인증·디스크 등 20종 이상 장애 유형을 자동 분류하고 원인·조치를 근거와 함께 추천

GOAL 3

사람이 통제하는 안전한 조치

운영자 승인과 서버 검증 없이는 어떤 명령도 실행되지 않는 실행 체계

GOAL 4

현업 적용 가능한 운영 수준

실서버 배포, 자동 검증, 민감정보 보호까지 갖춘 “바로 써볼 수 있는” 완성도

4개월	5명	99건	158개
2026.07 – 2026.10	코드 기여자	master 커밋 (월 26·37·19·17)	자동 검사 항목

① 목표달성도 달성 결과는 03 해결 마지막에 목표별로 보고합니다

02

문제

빠르게 대응하려면 위험하고,
안전하게 대응하려면 느립니다.

02 문제 4단계마다 발생하는 구조적 문제

장애 대응을 늦추고 위험하게 만드는 네 가지 문제

탐지

P1. 알림 폭주와 노이즈

장애 하나가 핵심 오류 뒤에 “read-only mode 전환”, “쓰기 실패” 같은 **후속 ERROR**를 줄줄이 남깁니다. 반복 오류까지 개별 장애로 쌓여 진짜 원인이 묻힙니다.

분석

P2. 경험에 갇힌 원인 분석

“지난번 같은 오류는 커넥션 풀 문제였다”는 지식이 **담당자 머릿속**에만 있습니다. 처음 보는 오류는 로그만으로 원인을 좁히기 어렵고, 업무 영향도 판단도 제각각입니다.

조치

P3. 위험한 수동·자동 조치

엉뚱한 호스트 재시작, 같은 명령의 **중복 실행**, 이중화가 없는 단일 인스턴스 재시작은 장애를 키웁니다. AI 추천을 그대로 실행하면 책임 소재도 사라집니다.

확인 · 보안

P4. 복구 미검증과 정보 유출

“명령 성공 = 복구”로 오판해 장애가 재발합니다. 또한 로그에 섞인 **주민번호·카드·계좌·토큰**이 외부 LLM으로 전송될 위험이 있어 AI 도입 자체가 막힙니다.

02 문제 문제 정의

우리가 풀어야 할 질문은 하나입니다

어떻게 하면 AI의 속도로 원인을 찾고, 사람의 통제 아래에서 검증된 복구까지 끝낼 수 있을까?

요구 조건	설계 원칙
빠르게	수집 즉시 비동기 분석, 반복·후속 오류는 묶어서 한 번만 분석
근거 있게	과거 사례 + 진단 결과 + 자원 지표를 근거로 제시하고, 확인되지 않은 원인은 단정하지 않음
안전하게	운영자 승인 없이는 실행하지 않음, 대상·버전·만료·중복 실행을 서버가 검증
끝까지	명령 완료와 업무 복구 확인을 분리하고, 해결 사례를 다시 학습 데이터로 축적

03

해결 — Hanati Jarvis

수집 → 분석 → 추천 → 승인 → 실행 → 복구 확인 → 학습
장애 대응 전 과정을 하나의 닫힌 루프로 연결합니다.

03 해결 솔루션 개요

문제 4개에 핵심 기능 5개로 답합니다

수집
Fluent Bit
Collector(psutil)

›

집약·분류
21종 오류 코드
P1~P3 우선순위

›

분석·추천
RAG + LLM
자원 가설 엔진

›

운영자 승인
Human-in-
the-loop

›

안전 실행
정책 Agent
등록 Runbook

›

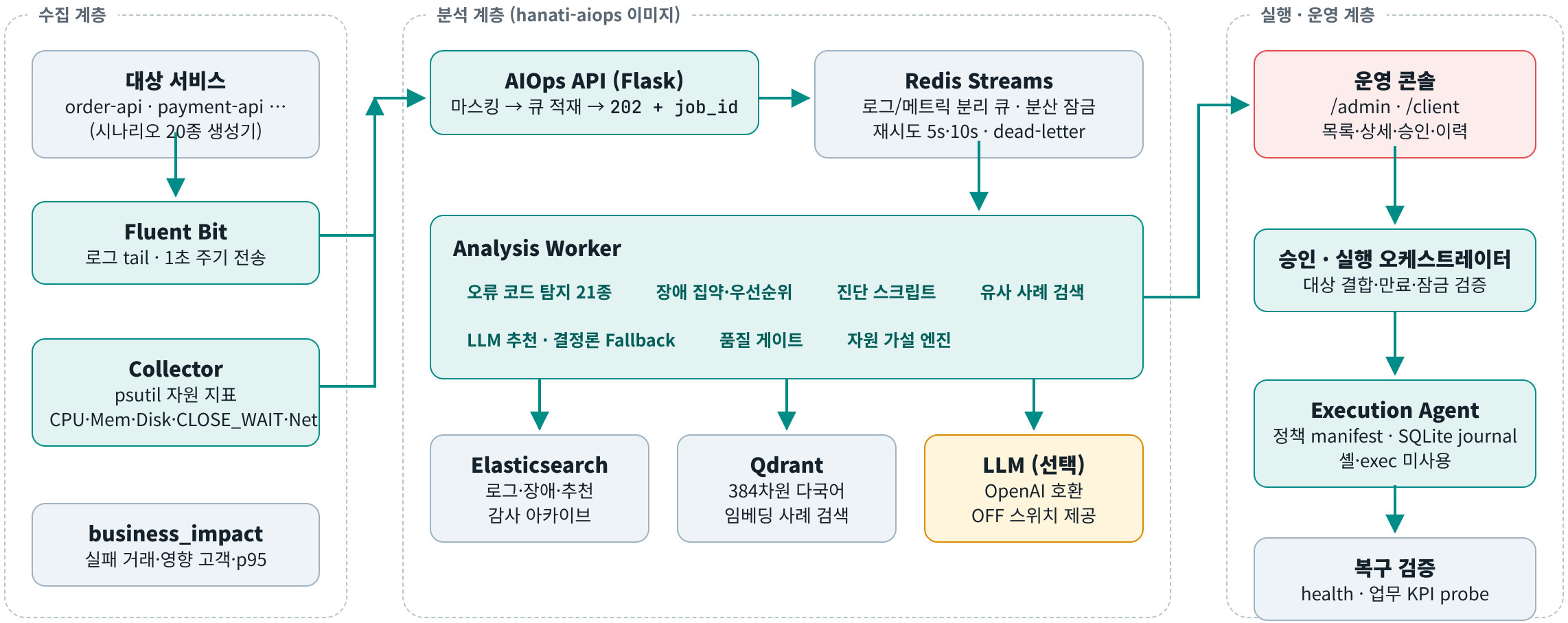
복구 확인
health + 업무 KPI
2회 검증

›

학습
확정 사례
Qdrant 승격

#	핵심 기능	해결하는 문제	한 줄 요약
F1	**지능형 장애 집약 · 업무 우선순위**	P1 알림 폭주	후속·반복 오류를 하나의 장애로 묶고 업무 영향으로 P1~P3 부여
F2	**하이브리드 원인 분석 (RAG + LLM + 규칙)**	P2 경험 의존	유사 사례 검색 + 진단 결과로 원인·조치 순위 추천, 품질 미달 시 자원 가설로 전환
F3	**승인 기반 안전 실행**	P3 위험한 조치	대상 결합·만료·분산 잠금·이중화 검사를 통과해야만 실행
F4	**복구 검증 · 피드백 학습 루프**	P4 복구 미검증	명령 완료와 업무 복구를 분리 검증, 확정 사례를 지식으로 축적
F5	**금융권 보안 · 거버넌스**	P4 정보 유출	저장·LLM 입력·응답 전 구간 민감정보 마스킹, 외부 LLM 차단 스위치, 감사 기록

Docker Compose 한 번으로 뜨는 이벤트 기반 AIOps 파이프라인



④ 산출물완성도 ⑤ 난이도 8GB / 2vCPU 단일 서버 기준 상시 메모리 상한 5.5GiB

03 해결 · F1 지능형 장애 집약 · 업무 우선순위

로그 100줄이 아니라 장애 1건으로 보여줍니다

F1

202 — **비동기 수집** — API는 민감정보를 마스킹해 Redis Streams에 넣고 즉시 202 + 작업 ID를 반환. 분석은 별도 Worker가 처리해 수집이 막히지 않습니다.

21종 — **오류 코드 자동 분류** — DISK_FULL, DB_DEADLOCK, CONNECTION_POOL_EXHAUSTED, PAYMENT_GATEWAY_FAILURE, ORA-28040 등 금융 시스템 빈발 장애를 패턴으로 식별합니다.

39개 — **후속 메시지 제외** — “Service entering read-only mode.” 같은 시나리오 후속 ERROR는 원문 로그로만 보존하고 별도 장애를 만들지 않습니다.

300초 — **반복 분석 재사용** — 같은 환경·서비스·정규화 메시지·오류 코드는 기존 장애에 발생 수만 누적. 유효한 추천이 있으면 LLM을 다시 호출하지 않아 비용·소음을 줄입니다.

업무 영향 기반 P1~P3

등급	판정 기준
P1	업무 중단(service_available=false) 또는 긴급 기준 초과
P2	tier1 핵심 업무, 실패 거래·영향 고객 발생
P3	업무 영향 미확인

“횟수만으로 업무 중단을 확정하지 않는다” — 보고된 피해 지표를 근거로만 등급을 올립니다.

콘솔은 최근 10분 장애를 진행중·긴급·미확인·분석중 탭으로 보여주고, 1분 이내 발생 건은 빨간 테두리로 강조합니다.

03 해결 · F2 하이브리드 원인 분석

베테랑의 경험을 검색 가능한 지식으로 바꿉니다

F2

① 진단 실행
오류 코드별 등록된
진단 스크립트 결과

›

② 유사 사례 RAG
Qdrant 다국어 임베딩
Top-3 과거 장애

›

③ LLM 추천
원인 + 조치 Runbook
추천도 순 랭킹(JSON)

›

④ 품질 게이트
후보 없음·빈 응답·
낮은 신뢰도 차단

›

⑤ 자원 가설 엔진
15분 지표로 증거 기반
가설·추가 진단 제시

LLM이 없어도 동작

API 키가 없거나 응답이 JSON이 아니면 **결정론적 랭커**가 규칙 순서 + 과거 사례 유사도로 조치 순위를 만듭니다. 어떤 환경에서도 추천 리스트는 비지 않습니다.

모르면 모른다고 말하기

미등록 오류나 신뢰도 미달은 원인을 지어내지 않고, CPU 포화·메모리 누수·디스크·CLOSE_WAIT·네트워크 오류 증가 등 **지표 근거가 있는 가설**과 다음 진단만 제시합니다.

로그 없이도 선제 탐지

메트릭 이상 탐지기가 메모리 증가 추세·디스크 임계치·커넥션 누수를 감지하고, 최근 10분 오류 로그와 **상관 분석**해 장애를 미리 생성합니다.

```
"error_code": "CONNECTION_POOL_EXHAUSTED", "priority": "P2",
"cause": "과거 유사 사례 기준 원인은 '커넥션 미반환으로 인한 풀 고갈'로 추정됩니다.",
"runbooks": [ {"script_id": "increase_connection_pool_size", "confidence": 0.95, "reason": "과거 유사 사례 3건(최고 유사도 0.87) 기반"}, … ]
```

④ 핵심기능 F2 → 문제 P2(경험 의존 분석) 해결 · 추천 출력 예시는 화면 표시 형식을 축약한 것

03 해결 · F3 승인 기반 안전 실행

AI는 제안하고, 사람이 승인하고, 시스템이 검증합니다

F3

대상 선택
추천에 연결된 환경·서비스·
호스트·인스턴스

›

승인 후 실행
운영자 클릭 1회
(승인자 기록)

›

서버 검증
8가지 실행 조건
실패 시 사유 표시

›

명령 실행
Agent 실행 직전
조건 재확인

›

MONITORING
복구 확인 전까지
해결로 보지 않음

실행 전 서버가 확인하는 조건
① 장애·추천 ID / 버전 / 유효 시간 일치 (변경·만료 시 차단)
② 대상 환경·서비스·호스트·인스턴스 결합 검증
③ Agent 인증 토큰(32자 이상) · 정책 해시
④ 승인된 Compose 프로젝트 · 컨테이너 · 이미지
⑤ 이중화 — 건강한 peer가 있어야 재시작 허용
⑥ 점검 라벨 · ⑦ 실행 전 상태 복원 가능성
⑧ Redis 분산 잠금 — 같은 대상 동시·중복 실행 차단

사고를 만들지 않는 실행

- **셸·호스트 명령·컨테이너 exec 미사용** — Docker API의 조회/재시작 내장 작업만 허용
- **SQLite journal** — 응답 유실 시 재실행하지 않고 원래 실행 기록을 조회
- 재시작 실패 시 동일 이미지의 **실행 전 상태로 자동 복원**, 불확실한 결과는 잠금 유지
- Runbook 58개는 **SHA-256**으로 등록, 파일 변경 시 실행 차단

대응 스크립트가 없는 장애는 **수동 조치 등록**으로 방법·운영자·복구 여부를 이력화합니다.

④ 핵심기능 F3 → 문제 P3(위험한 조치) 해결

03 해결 · F4 복구 검증 · 피드백 학습 루프

“재시작 성공”이 아니라 “업무 정상화”를 확인합니다

F4

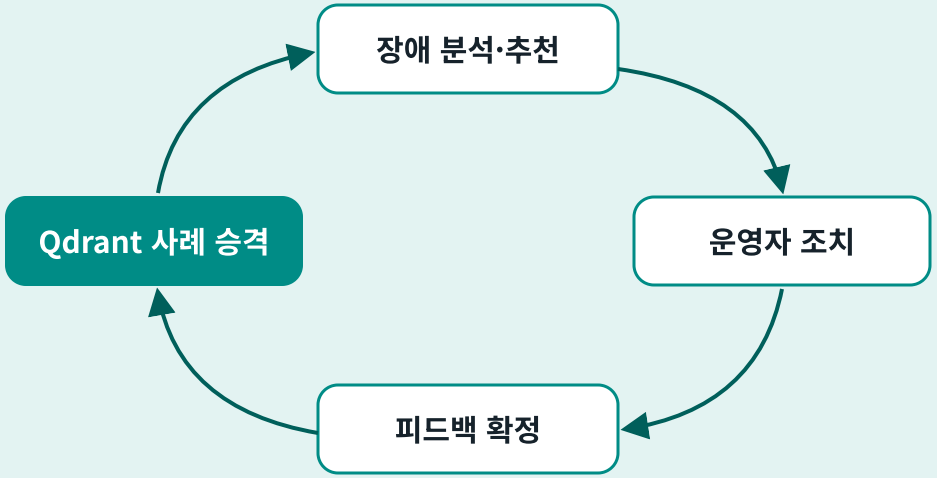
2단계 상태 모델

상태	의미
MONITORING	명령 또는 수동 조치 완료 · **복구 확인 대기**
RESOLVED	복구 검사 또는 운영자 복구 확인 **완료**

읽기 전용 업무 KPI Probe

조치 **완료 이후** 시작된 집계 구간의 성공률·실패율·p95 응답시간·거래 수를 업무 상태 API에서 GET으로 조회합니다. 컨테이너 health와 함께 **2회 연속 통과**해야 RESOLVED로 전환됩니다. 허용 URL 목록·리다이렉트 차단·HTTPS 기본.

쓸수록 똑똑해지는 플라이휠



운영자가 **confirmed + 복구됨 + 확정 원인 + 성공 조치**를 모두 입력한 피드백만 새 사례로 승격합니다. 검증되지 않은 정보로 지식베이스가 오염되지 않습니다.

03 해결 · F5 금융권 보안 · 거버넌스

AI를 쓰되, 고객 정보는 밖으로 나가지 않게

F5

```
# 원문 로그
payment failed rrn=900101-1234567 card=4111 1111 1111 1111
Authorization: Bearer eyJhbGciOi...  password=p@ss

# 저장 · LLM 입력 · API 응답 (전 구간 동일 적용)
payment failed rrn=[RRN] card=[CARD_OR_ACCOUNT]
Authorization: Bearer [REDACTED]  password=[REDACTED]
```

마스킹 대상

주민번호 · 카드/계좌 · 전화 · 이메일 · 비밀번호 · 토큰 · API 키 · URL 자격증명

LLM OFF 스위치

`LLM_EXTERNAL_ENABLED=false` 한 줄로 외부 호출 차단, 결정론 추천으로 전환

감사 가능한 운영

- 모든 승인·실행·수동 조치에 **운영자 · 시각 · 방법 · 확인 방식** 기록
- 실행 ID 재사용·중복 요청은 기존 결과 반환 (멱등성)
- 감사 스트림 90일 · dead-letter 30일 경과분은 Elasticsearch 아카이브로 이관
- 화면 출력 HTML 이스케이프, 시나리오 데이터는 `synthetic=true`로 실데이터와 분리
- 실행 Agent는 비밀값·정책 파일을 Git·이미지에 포함하지 않음

> **보안팀이 “쓰지 말라”가 아니라 “이렇게 쓰면 된다”고 말할 수 있는 구조.**

03 해결 · DEMO 시연 시나리오 — 결제 서비스 DB 커넥션 풀 고갈

관리자 화면에서 장애를 발생시키고 해결까지 따라가 봅니다

1. **시나리오 실행** — /admin에서 20개 장애 시나리오 중 하나를 선택, Redis로 실행을 공유해 /client도 1초 주기로 추종
2. **탐지 · 집약** — Fluent Bit 1초 수집 → 장애 1건 생성, '최근 1분' 빨간 강조 · 후속 ERROR는 장애로 만들지 않음
3. **분석 결과** — 상세에서 단위시스템·호스트·원문·최근 로그와 함께 원인 추정, 유사 사례, 추천 Runbook 순위 확인
4. **승인 후 실행** — 대상 선택 → '승인 후 실행' → 등록 스크립트 실행 결과 표시 → 상태 MONITORING
5. **복구 확인 · 이력** — '업무 복구 확인' 통과 시 처리 완료, 다른 세션 화면에도 즉시 동기화 · Redis/Worker 처리 로그로 경로 추적

시연 포인트 — **로그 수십 줄이 장애 1건으로, 추천 근거(유사 사례·진단)가 화면에, 승인 없이는 아무것도 실행되지 않음, 복구 확인 전까지는 완료가 아님.**

④ 핵심기능 시연 Runbook 매핑: 시나리오 20개 중 19개 자동 연결 · RATE_LIMIT_EXCEEDED는 수동 조치 등록

03 해결 기존 서비스 연계 · 업무 개선

기존 자산은 그대로 살리고, 운영 업무는 실질적으로 바꿉니다

기존 자산	Jarvis 연계 방식 (시너지)
애플리케이션 로그	Fluent Bit tail로 **업무 코드 수정 없이** 수집
기존 운영 스크립트	SHA-256으로 Runbook 등록 → 승인 후 실행 (58개 재활용)
업무 상태·KPI API	읽기 전용 GET probe로 **복구 판정**에 활용
Docker·Compose 환경	정책 기반 Execution Agent가 조회·재시작 대행
LLM · 검색 인프라	포트·어댑터 구조로 사내 LLM·저장소 교체

업무	Before	After (Jarvis)
인지	로그 검색 도구에서 수동 확인	최근 10분 장애 목록 · 1분 내 발생 강조
분석	담당자 경험·기억	유사 사례 + 진단 결과 근거로 추천
조치	서버 접속 · 수동 명령	콘솔에서 대상 선택 · 승인 1회
확인	“재시작 성공 = 복구”	health + 업무 KPI 2회 확인
기록	사후 수기 보고	운영자·시각·방법 자동 이력화

새 시스템을 “하나 더” 들이는 것이 아니라, 이미 있는 **로그 · 스크립트 · 업무 API**를 엮어 장애 대응 흐름을 완성합니다.

03 해결 CoP 목표 대비 달성도

네 가지 목표, 계획한 범위를 모두 구현했습니다

목표	달성 결과 (master 기준)	상태
G1 End-to-End 파이프라인	수집 → 집약 → 분석·추천 → 승인 → 실행 → 복구 확인 → 학습 **7단계 루프**, Compose **11개 서비스** 명령 한 줄 기동	달성
G2 자동 분석·추천	오류 **21종** 자동 분류, 시나리오 **20개 중 19개** Runbook 자동 연결, RAG + LLM + 자원 가설 엔진, LLM 없이도 추천 유지	달성
G3 안전한 조치	승인 시 서버가 **8가지 조건** 검증, 분산 잠금·실행 기록으로 중복 실행 차단, 대상 없는 장애는 수동 조치 이력화	달성
G4 현업 적용 수준	GitHub Actions → **EC2 자동 배포**, 8GB·2vCPU 자원 설계, 전 구간 마스킹 · 자동 검사 **158개 중 157개 통과**(1개는 Docker 환경 필요로 생략) 실제 거래 KPI 연동 · 로그인/RBAC는 운영 환경 정보가 필요해 Phase 2로 이관	핵심 달성

21종
자동 분류 오류 유형

58개
등록 Runbook 스크립트

157/158
자동 검사 통과

5.5GiB
상시 메모리 상한

목표를 “기능 목록”이 아니라 **운영자가 실제로 쓰는 흐름**으로 정의했고, 각 목표의 결과를 자동 검사와 실서버 배포로 확인했습니다.

① 목표달성도 ④ 산출물완성도 출처: 저장소 INTEGRATION_RESULTS.md · OPERATIONS.md

03 해결 기술 난이도 · 현업 적용 가능성

어려운 문제를 풀었고, 현장에 바로 놓을 수 있습니다

도전 수준 — 이번 CoP에서 해결한 기술 과제

- **비동기 분산 처리** — Redis Streams 큐, 재시도·dead-letter, Worker 생존 신호
- **하이브리드 AI** — 다국어 임베딩 RAG + LLM + 결정론 Fallback + 품질 게이트
- **멱등 실행** — 실행 ID·분산 잠금·SQLite journal로 응답 유실에도 재실행 없음
- **세션 간 실시간 동기화** — 관리자·클라이언트 화면, 지연 응답 덮어쓰기 방지
- **자원 제약 운영** — ES·Qdrant·임베딩 모델을 8GB·2vCPU 한 대에 배치

실행 가능성 — 현업 적용 조건

- **설치** — `docker compose up -d --build` 한 줄
- **망분리·폐쇄망** — `LLM_EXTERNAL_ENABLED=false`로 외부 호출 없이 동작
- **업무 시스템 무변경** — 로그 tail 수집, 읽기 전용 KPI 조회
- **기존 스크립트 재활용** — 등록만으로 승인 실행 대상 편입
- **검증 도구 내장** — `deployment-check`, 10/50/100건 `load_check`
- **실배포 경험** — master push 시 EC2 자동 배포

난이도는 높게, 도입 장벽은 낮게 — **업무 시스템을 바꾸지 않고, 외부망 없이도, 서버 한 대로** 시작할 수 있습니다.

04

스케일업

단일 서버 1단계에서,
실 업무 · 멀티 클러스터 · 그룹 공통 플랫폼으로.

04 스케일업 단계별 확장 로드맵

이미 준비된 확장 지점에서 한 단계씩 넓혀갑니다

PHASE 1 · 현재

단일 서버 PoC

- Compose 11개 서비스
- 21종 오류 · 20개 시나리오
- 승인 기반 실행 · 복구 검증
- EC2 자동 배포

PHASE 2

실 업무 연동

- 실제 거래 KPI probe 연결 (결제·계정계)
- SSO · RBAC 승인 권한 분리
- 메신저·온콜 알림 연동
- 조직별 마스킹 · 우선순위 정책

PHASE 3

규모 확장

- Kubernetes · 멀티 클러스터 Agent
- Worker 수평 확장 · P1 우선 큐
- 사내 LLM / 온프레미스 모델 전환
- ES · Qdrant 클러스터 구성

PHASE 4

그룹 공통 플랫폼

- 관계사 공통 AIOps 허브
- 관계사 간 장애 사례 공유
- 검증된 저위험 조치의 정책 기반 자동 승인
- 장애 예측 · 용량 계획

로드맵은 현재 코드가 이미 분리해 둔 경계(포트·어댑터, 정책 파일, Agent HTTPS 원격 대상, 분리된 큐)를 그대로 확장하는 방향으로 설계했습니다.

04 스케일업 임팩트 · 확산성

한 팀의 성과를 조직 전체의 자산으로

CoP 팀
PoC 구축 · 시나리오 20종
운영 가이드 작성

›

운영 조직
담당 서비스 등록
실 업무 KPI 연동

›

전사 운영
공통 장애 지식베이스
표준 대응 절차

›

그룹 관계사
공통 AIOps 허브
관계사 간 사례 공유

① 지식이 확산됩니다

운영자가 확정한 장애 사례가 Qdrant에 쌓여 **다른 팀의 장애 분석에도 바로 쓰입니다**. 개인 노하우가 조직 자산이 됩니다.

② 설치가 확산됩니다

Compose 한 줄, 8GB 서버 기준 자원 설계 그대로 **다른 팀이 동일하게 설치**할 수 있습니다.

③ 온보딩이 쉽습니다

새 서비스는 오류 규칙 · 실행 대상 · KPI 정책 **파일 등록만으로 편입**됩니다. 코드 수정이 거의 필요 없습니다.

④ 경험이 공유됩니다

README · ARCHITECTURE · OPERATIONS · TESTING 문서와 자동 검사로 이관이 쉽고, **“금융권에서 AI를 안전하게 쓰는 설계”**를 사내에 공유합니다.

바꿔 끼우기 쉬운 구조, 숫자로 증명할 성과

확장 지점	현재 구조
LLM 교체	RecommendationGenerator 포트 — OpenAI 호환 / 사내 모델 / 결정론 랭커
저장소 교체	LogRepository · CaseSearcher 포트 — ES · Qdrant 어댑터
대상 추가	execution-targets · agent manifest 정책 파일 등록만으로 확장
오류 유형 추가	ERROR_RULES에 패턴 · 진단 · 조치 후보 등록
업무 검증 추가	business_probe 플러그인 (http_kpi)

도입 후 측정할 핵심 지표

MTTR 장애 발생 → RESOLVED까지 시간 (상태 이력으로 자동 산출)

노이즈 원시 ERROR 대비 생성 장애 수 · 반복 분석 재사용률

채택률 1순위 추천 Runbook이 실제 승인·복구로 이어진 비율

재발률 RESOLVED 이후 REOPENED 비율 · 사례 승격 건수

모든 지표는 Jarvis가 이미 저장하는 **장애 상태 이력 · 발생 수 · 승인/실행 기록 · 운영자 피드백**에서 바로 계산할 수 있어, 도입 효과를 별도 수작업 없이 정량 보고할 수 있습니다.

SUMMARY

AI의 속도로 찾고, 사람의 통제로 실행하고, 업무 지표로 끝까지 확인합니다.

① **목표달성도**	4개 목표 중 3개 완전 달성, 1개 핵심 달성 (실 업무 연동은 Phase 2)
② **회사성장기여도**	기존 로그·스크립트·업무 API를 연결해 장애 대응 5단계 업무를 개선
③ **임팩트/확산성**	사례 지식베이스 · 표준 설치 · 파일 등록 온보딩으로 팀 → 그룹 확산
④ **산출물완성도**	핵심 기능 5개 · 시연 · 자동 검사 157/158 통과 · EC2 실배포
⑤ **난이도·실행가능성**	분산 처리 · 하이브리드 AI · 멱등 실행 / 폐쇄망 · 무변경 연동

감사합니다 · Q&A